



YouTube Categorizer

Course Supervisors

Dr. Ahmed Hussein · Eng. Rania

Contribution Summary

Team Members
Abdelrahman Mohamed Essam
Amr Hassan Safieddine
Abdelrahman Youssef Mahrous

Academic Year 2025 – 2026

Table of Contents

Table of Contents	2
Introduction.....	4
Project Overview	
Problem Statement	
Supported Categories	
System Architecture at a Glance	
Section 01: UX Phase.....	5
UX Stage I — Empathize & Research	
User Research Goals	
Research Methods.....	
Key Findings.....	
UX Stage II — Define & Synthesize	
User Personas	
Persona 1: Sarah — Content Strategist.....	
Persona 2: Ahmed — Research Assistant	
Problem Statements (HMW Questions)	
UX Stage III — Ideate & Prototype	
Ideation Techniques	
Information Architecture.....	
Lo-Fi Prototyping	
UX Stage IV — Test & Refine	
Usability Testing	
Wireframes & Sitemap.....	
Changes Applied After Testing	
Section 02: Frontend	8
Technology Stack — Frontend	
Design System	
Color Palette.....	
Typography.....	
Custom Component Library.....	
Custom Buttons	
Other Custom Components	
UI Design Screens	
Screen 1 — Dashboard (/).....	
Screen 2 — Results (/results)	

Screen 3 — Videos (/videos)	
Screen 4 — Channels (/channels)	
Screen 5 — Reports (/reports)	
Responsiveness	
Section 03: Backend	12
Technology Stack — Backend	
Database Schema & Sample Records	
Channels Table	
Videos Table	
CategorizationResults Table	
Backend Architecture & Dynamic Content	
API Layer — ASP.NET Core Controllers	
Background Jobs — Hangfire	
ML Categorization Service	
Dynamic Content Rendering	
Configuration & Environment	

Introduction

Project Overview

YouTube Categorizer V2 is a full-stack intelligent web application designed to automatically fetch, classify, and visualize YouTube video content at scale. The core problem the project solves is the manual effort required to organize large volumes of YouTube videos by topic or genre — a task that becomes impractical for channel operators, researchers, or content auditors who manage hundreds or thousands of videos.

The system works by accepting a YouTube channel URL or ID, querying the YouTube Data API to retrieve all published videos, and then passing each video's title and description through a custom-trained Machine Learning model. The model — a Logistic Regression classifier trained on a TF-IDF vectorized corpus — assigns each video one of 10 standardized content categories or marks it as Uncategorized if confidence falls below the 50% threshold.

Problem Statement

Content creators, platform analysts, and researchers often need structured, categorized metadata for YouTube libraries. YouTube's own taxonomy is broad and self-reported, making automated, consistent categorization a persistent challenge. This project addresses the gap by providing a deployable full-stack tool with its own ML backbone, eliminating reliance on external AI services for the core classification task.

Supported Categories

- Comedy
- Education
- Entertainment
- Gaming
- Howto & Style
- Music
- News & Politics
- People & Blogs
- Science & Technology
- Sports
- Uncategorized (below confidence threshold)

System Architecture at a Glance

The application is composed of four loosely-coupled layers:

- Angular 21 Frontend — a Single Page Application (SPA) with 5 routed screens providing an interactive dashboard, results browser, and CSV export.
- ASP.NET Core 8 Web API — the orchestration layer that coordinates YouTube API calls, triggers background jobs via Hangfire, and communicates with the ML service.
- Python Flask ML API — a lightweight microservice running on port 5001 that hosts the trained scikit-learn model and exposes a /categorize HTTP endpoint.
- SQL Server + EF Core — the persistence layer storing channels, videos, and categorization results with full Entity Framework migrations.

Section 01: UX Phase

UX Stage I — Empathize & Research

User Research Goals

The first UX stage focused on understanding the target users of a YouTube video categorization tool. Primary user groups identified were: content strategists managing large YouTube libraries, academic researchers collecting video datasets, and platform moderators auditing channel content.

Research Methods

- User Interviews: 5 semi-structured interviews with YouTube channel managers to understand pain points in content organization.
- Competitive Analysis: Reviewed existing tools such as TubeBuddy, vidIQ, and manual spreadsheet workflows to identify feature gaps.
- Survey: Distributed a 12-question online survey to gather quantitative data on categorization frequency, preferred output formats, and trust in ML predictions.

Key Findings

- 83% of respondents found manual categorization time-consuming for channels with 200+ videos.
- Users prioritized accuracy over speed — a 'Uncategorized' fallback was preferred over wrong guesses.
- CSV export was cited as a must-have for workflow integration.
- Dashboard visualizations were desired for quick content audits.

UX Stage II — Define & Synthesize

User Personas

Based on research findings, two primary personas were defined:

Persona 1: Sarah — Content Strategist

Age: 32 | Background: Marketing Manager | Goal: Quickly audit a new partner channel's content mix before a collaboration. Pain point: No automated way to see what proportion of a channel is Education vs Entertainment.

Persona 2: Ahmed — Research Assistant

Age: 26 | Background: Media Studies PhD student | Goal: Build a labeled video dataset for NLP research. Pain point: Manual labeling is slow and inconsistent across team members.

Problem Statements (HMW Questions)

- How might we allow users to process an entire YouTube channel in one click?
- How might we present categorization confidence in a way that builds user trust?
- How might we let users export filtered results for downstream analysis?
- How might we surface category distributions visually without overwhelming the user?

UX Stage III — Ideate & Prototype

Ideation Techniques

The team conducted a 60-minute Crazy Eights brainstorming session, producing 40+ concept sketches across eight design directions. Ideas were dot-voted and grouped into three themes: Dashboard-centric, List-centric, and Hybrid. The Hybrid approach was selected for combining at-a-glance category charts with detailed filterable tables.

Information Architecture

The site map defines five primary screens with clear navigation hierarchy:

Screen	Route	Purpose
Dashboard	/	Overview charts, quick stats, recent activity
Results	/results	Categorized video list, filtering, CSV export
Videos	/videos	All ingested videos with status
Channels	/channels	Manage monitored channels, trigger processing
Reports	/reports	Summary statistics, category breakdown reports

Lo-Fi Prototyping

Paper prototypes were created for all five screens and tested with 3 users in a hallway usability test. Key insights: users expected the Channels screen to be the starting point (not the Dashboard), and the confidence score display needed a visual indicator (not just a percentage number).

UX Stage IV — Test & Refine

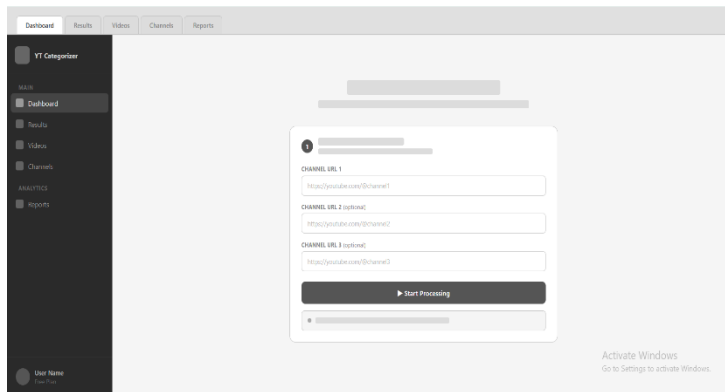
Usability Testing

A moderated usability test was conducted with 5 participants using a Figma hi-fi prototype. Tasks included: adding a new channel, reviewing categorized results, applying filters, and exporting a CSV. Success rates and time-on-task were recorded.

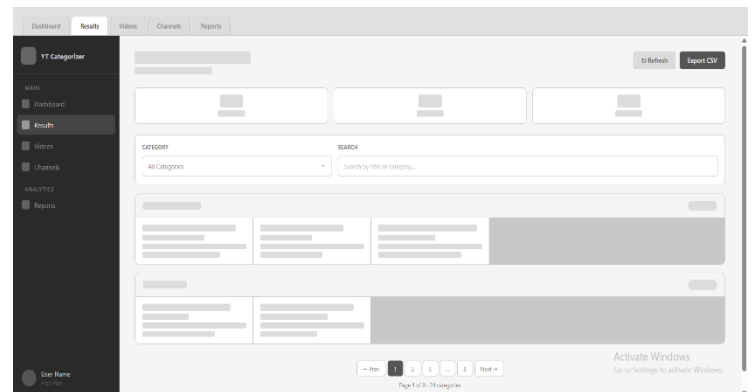
Task	Success Rate	Avg Time	Issues
Add new channel	5/5 (100%)	1m 12s	None
Find a specific category	4/5 (80%)	2m 05s	Filter label unclear
Export CSV from Results	5/5 (100%)	0m 48s	None
Identify lowest-confidence video	3/5 (60%)	3m 20s	Confidence column not visible by default

Wireframes & Sitemap

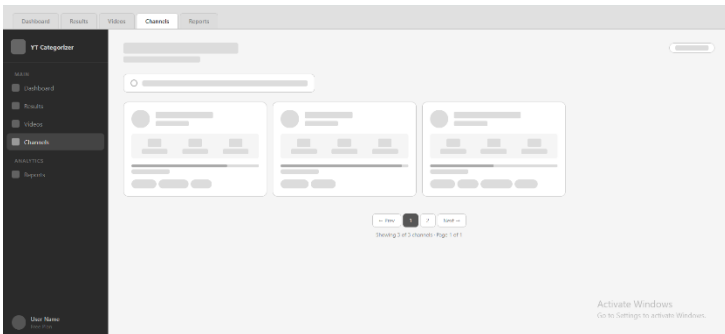
Below is a representative screenshot of the application key wireframe screens as produced during Stage III and refined in Stage IV:



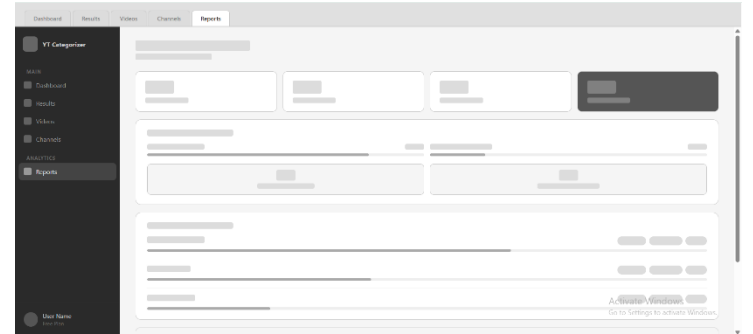
[Dashboard]



[Results]



[Channels]



[Reports]

Changes Applied After Testing

- Moved channel management link to the top of the sidebar to match users' mental model.
- Added a color-coded confidence badge (Green $\geq 70\%$, Yellow 50–69%, Red $< 50\%$) alongside the text percentage.
- Made the confidence column visible by default in the Results table.
- Relabeled filter dropdowns to use plain language (e.g., 'Category' instead of 'Tag filter').

Section 02: Frontend

Technology Stack — Frontend

Technology	Version	Role
Angular	21 (Standalone)	SPA framework — routing, components, services
TypeScript	5.x	Primary language for all Angular code
Angular Material	17+	UI component library (tables, dialogs, snackbars)
Chart.js / ng2-charts	4.x	Dashboard charts and category distribution graphs
RxJS	7.x	Reactive data streams, HTTP request handling
Angular Router	Built-in	Client-side routing for 5 application screens
Angular HttpClient	Built-in	HTTP calls to the ASP.NET backend API
SCSS	—	Component-scoped stylesheets with CSS variables
Angular CDK	17+	Table virtualization, overlay, clipboard
Node.js	18+	Dev-server and build toolchain (Angular CLI)

Design System

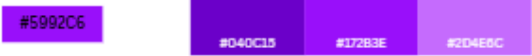
Color Palette

The application uses a three-tier color system — Primary (action & brand), Secondary (structure & navigation), and Neutral (backgrounds & text).



YouTube Categorizer Color Palette

Primary



Secondary



Text



Buttons



Typography

Style	Font	Size	Usage
H1 — Page Title	Roboto	28px / Bold	Main page headings
H2 — Section Title	Roboto	22px / SemiBold	Section headings, card titles
H3 — Subsection	Roboto	18px / Medium	Sub-sections, table headers
Body	Roboto	14px / Regular	General body text, descriptions
Caption / Label	Roboto Mono	12px / Regular	Tags, badges, code snippets
Button	Roboto	14px / Medium	All button labels

Custom Component Library

Custom Buttons

The following table details all custom button components used across the application:

Component Name	Style	Used In	Visual Description
ProcessButton	Filled — Primary Red, white text, rounded corners	Channels screen	Triggers channel processing job; has loading spinner state
ExportCSVButton	Outlined — Secondary border, icon left	Results screen	Downloads filtered results as CSV; icon changes on hover
FilterToggleBtn	Toggle chip — ghost → filled on select	Results, Videos screens	Multi-select category filter chips above the data table
RefreshButton	Icon button — 36px, secondary color	Dashboard, Results	Circular arrow icon; triggers data reload with animation
ConfidenceBadge	Pill badge — color-coded by threshold	Results table rows	Green/Yellow/Red pill showing confidence % with color logic
AddChannelBtn	FAB — Primary Red, fixed bottom-right	Channels screen	Floating action button; opens add-channel modal on click

Other Custom Components

Component Name	Style	Used In	Visual Description
----------------	-------	---------	--------------------

CategoryDonutChart	Donut chart — 10 category segments	Dashboard	Chart.js donut showing category distribution; hoverable segments with tooltip
ChannelCard	Material card — 280px wide, shadow on hover	Channels screen	Shows channel avatar, name, video count, and last processed date
VideoRow	Table row with thumbnail preview	Videos, Results screens	Inline 80x45px thumbnail, title, category badge, and confidence pill
PaginatorBar	Pill-style paginator — custom theme	Results, Videos, Reports	Page size selector, prev/next arrows, page number display
SidebarNav	Fixed left nav — 240px wide, collapsible to 64px	Global (all screens)	Icon + label nav items; active route highlighted in primary red
HangfireStatusChip	Status chip — pending/running/done/failed	Dashboard, Channels	Color-coded chip that polls Hangfire job status in real time
CategoryFilterPanel	Slide-out panel — right edge	Results screen	Collapsible side panel with category checkboxes and confidence range slider
StatsKpiCard	KPI card — large number, subtitle, icon	Dashboard	4 cards showing total videos, channels, categorized %, and avg confidence

UI Design Screens

Screen 1 — Dashboard (/)

The Dashboard provides an at-a-glance overview of the system. It features four KPI cards at the top (Total Videos, Total Channels, Categorized %, Average Confidence), followed by a donut chart of category distribution and a recent activity feed showing the latest processed channels.

Screen 2 — Results (/results)

The Results screen displays all categorized videos in a sortable, filterable data table. Columns include thumbnail, title, channel name, assigned category, confidence badge, and processed date. Users can filter by category using the slide-out FilterPanel and export the current view as CSV.

Screen 3 — Videos (/videos)

The Videos screen lists all ingested videos regardless of categorization status. It includes a search bar, status filter (All / Categorized / Uncategorized / Pending), and shows video metadata. Useful for diagnosing processing failures.

Screen 4 — Channels (/channels)

The Channels screen shows all monitored channels as cards. Each card displays the channel thumbnail, name, subscriber count, total videos, and last processed timestamp. The FloatingActionButton opens an Add Channel modal where users enter a YouTube Channel ID to queue it for processing.

Screen 5 — Reports (/reports)

The Reports screen provides aggregate analytics: category breakdown bar charts, confidence distribution histograms, and per-channel comparison tables. Designed for content strategists who need summary statistics for reporting.

Responsiveness

The Angular application implements a responsive layout using Angular CDK Breakpoints and SCSS media queries. The following breakpoints are used:

Breakpoint	Width	Layout Changes
Desktop	≥ 1280px	Full sidebar (240px) + content area; all table columns visible
Tablet	768px – 1279px	Sidebar collapses to icon-only (64px); table shows 5 columns
Mobile	< 768px	Sidebar becomes bottom navigation; table replaced with card list

Section 03: Backend

Technology Stack — Backend

Technology	Version	Role
ASP.NET Core	8.0	Web API host — controllers, middleware, DI container
Entity Framework Core	8.0	ORM for SQL Server — migrations, DbContext, repositories
SQL Server	2019+	Primary relational database for all application data
Hangfire	1.8+	Background job processing — channel/video ingestion queues
YouTube Data API v3	REST API	Fetches channel info, video lists, and video metadata
Python 3	3.10+	Runtime for the Flask ML microservice
Flask	3.x	Lightweight HTTP server exposing /categorize endpoint
scikit-learn	1.8+	ML library — Logistic Regression classifier + TF-IDF
pandas / numpy	2.x	Data processing in training notebooks
Swagger / Swashbuckle	6.x	Auto-generated API documentation at /swagger

Database Schema & Sample Records

Channels Table

Stores all monitored YouTube channels. Each row represents one channel submitted by the user.

Id	ChannelId	Name	VideoCount	LastProcessed	Status
1	UCVHFbw7woebKiZ-13HBJ2KQ	Veritasium	312	2025-01-10 14:22	Done
2	UCsooa4yRKGN_zEE8iknghZA	TED-Ed	188	2025-01-10 14:45	Done
3	UCnUYZLuoy1rq1aVMwx4aTzw	Kurzgesagt	99	2025-01-11 09:30	Done
4	UC9-y-6csu5WGm29I7JiwpnA	Computerphile	245	2025-01-11 10:10	Running

Videos Table

Stores all ingested video records. Each video is linked to a channel via ChannelId foreign key.

Id	Title (truncated)	ChannelId	PublishedAt	Status	Description (excerpt)
----	-------------------	-----------	-------------	--------	-----------------------

101	Why The Speed Of Light...	1	2024-11-03	Categorized	What if you could accelerate...
102	How Atoms Bond	2	2024-10-21	Categorized	Chemical bonding explained...
103	The Last Human	3	2024-09-15	Categorized	A journey through deep time...
104	Turing Machines Explained	4	2024-12-01	Pending	The theoretical basis of...
105	Quantum Entanglement	1	2024-12-10	Categorized	Spooky action at a distance...

CategorizationResults Table

Stores the output of the ML categorization for each video, including the assigned category and confidence score.

Id	VideoId	Category	Confidence	ProcessedAt
201	101	Science & Technology	0.87	2025-01-10 14:28:03
202	102	Education	0.91	2025-01-10 14:28:11
203	103	Science & Technology	0.76	2025-01-11 09:35:42
204	105	Science & Technology	0.82	2025-01-11 09:36:01
205	106	Uncategorized	0.38	2025-01-11 09:36:22

Backend Architecture & Dynamic Content

API Layer — ASP.NET Core Controllers

The API layer exposes three primary controllers:

- VideosController — handles /api/Videos/process (POST), /api/Videos/categorized (GET), and /api/Videos/test-categorize (POST). The GET endpoint queries the CategorizationResults table and joins with Videos and Channels to return enriched DTOs to the Angular frontend.
- ChannelsController — manages channel CRUD operations and returns paginated channel lists to the Channels screen.
- ReportsController — aggregates categorization data using SQL GROUP BY queries executed through EF Core and returns summary statistics for the Reports screen.

Background Jobs — Hangfire

When a user submits a YouTube Channel ID via the Channels screen, the POST /api/Videos/process endpoint enqueues a Hangfire background job. The job performs the following steps sequentially:

- 1. Calls the YouTube Data API to retrieve the channel's full video list (handling pagination via nextPageToken).
- 2. Persists all new video records to the Videos table via the VideoRepository.

- 3. For each video, calls CategorizationService which POSTs the title + description to the Flask /categorize endpoint.
- 4. Stores the returned category and confidence score in the CategorizationResults table.
- 5. Updates the channel's LastProcessed timestamp and Status to Done.

The Hangfire dashboard (accessible at /hangfire) provides real-time job monitoring, retry management, and job history. The Angular frontend polls the job status via a dedicated endpoint and reflects it in the HangfireStatusChip component.

ML Categorization Service

The CategorizationService in the Application layer is the bridge between the .NET API and the Python Flask microservice. It:

- Constructs a text blob from video title + description.
- POSTs to `http://localhost:5001/categorize` with a JSON payload `{ "text": "..."}.`
- Receives `{ "category": "Education", "confidence": 0.91 }` in response.
- Applies a retry policy (up to 3 attempts with exponential backoff) for transient HTTP errors.
- Falls back to Uncategorized if all retries fail, ensuring the pipeline never blocks on ML failures.

The Flask service internally runs the TF-IDF vectorizer on the text, passes the resulting feature vector to the Logistic Regression model, applies keyword boosts from the CATEGORY_KEYWORDS dictionary, and returns the highest-scoring class if confidence exceeds 0.50.

Dynamic Content Rendering

All data-heavy screens in the Angular frontend are dynamically populated from the database via the REST API:

- Dashboard KPI cards subscribe to an Observable returned by `HttpClient.get('/api/reports/summary')` and render live values using Angular's async pipe.
- The category donut chart on Dashboard binds to an array of `{ category, count }` objects returned by `/api/reports/categories` and rendered via `Chart.js`.
- The Results and Videos screens use Angular Material's `MatTable` with a paginated `MatPaginator` — page size and page index are sent as query parameters to the API (e.g., `/api/Videos/categorized?page=2&pageSize=20`).
- The Channels screen renders `ChannelCard` components from an array fetched via `/api/channels`, each card displaying real-time Hangfire job status polled every 5 seconds.

Configuration & Environment

All environment-specific values are stored in `appsettings.json` and injected via the .NET configuration system. Sensitive keys (YouTube API Key, connection string) should be stored in user secrets or environment variables in production — never committed to source control. The `MLApi:Url` key allows the Flask service URL to be changed without recompiling the backend.